

CS 650 – Full Stack Application Develop

3-1 Short Paper: Front-End Framework Comparison

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

July 22, 2026

React and Vue Framework Comparison for an Internal Analytics Dashboard

Modern organizations increasingly rely on single-page applications (SPAs) to provide responsive, interactive, and scalable user experiences. Component-based frameworks have become the standard approach for building these applications because they allow developers to divide complex interfaces into reusable components that are easier to maintain and extend over time. For the proposed internal analytics dashboard, selecting an appropriate front-end framework is an important decision because it will affect development speed, maintainability, performance, and future scalability. This paper compares React and Vue, examines their shared component-based features, and recommends the framework that best fits the project requirements.

Scenario Constraints

Several factors influence the framework selection for this project. The application must display business metrics, interactive charts, and graphs while allowing employees to filter data by different time periods. Since the back-end services already exist, the front-end framework must efficiently consume REST APIs and dynamically update the interface without requiring full page reloads. The development team is newly formed and plans to deliver an initial version within a few months. Therefore, rapid development, ease of learning, and maintainability are important considerations. It is also reasonable to assume that team members have varying levels of experience with modern JavaScript frameworks. Because the dashboard is intended for internal company use rather than public-facing traffic, development efficiency and long-term maintainability are more important than handling extremely large user loads. The selected

framework should also support reusable components, efficient state management, and integration with visualization libraries.

Overview of React and Vue

React is an open-source JavaScript library developed by Meta for building interactive user interfaces. It is primarily written using JavaScript and JSX, although TypeScript is commonly used in professional applications. React focuses on building reusable UI components that manage their own state while efficiently updating only the portions of the page that change through its virtual Document Object Model (DOM) (React, 2025). React is commonly used to build single-page applications and large enterprise systems requiring scalable front-end architectures. Many well-known organizations use React, including Meta, Netflix, Airbnb, Shopify, and Dropbox. Its extensive ecosystem provides numerous third-party libraries for routing, state management, testing, and data visualization, making it suitable for complex enterprise applications. Vue is also an open-source JavaScript framework designed specifically for building user interfaces and single-page applications. Vue supports JavaScript, HTML templates, CSS, and TypeScript. Unlike React, Vue offers more built-in functionality, allowing developers to begin building applications with fewer additional libraries (Vue.js, 2025). Vue organizes applications into reusable components that combine templates, business logic, and styling into single-file components, improving readability and maintainability. Vue has been adopted by organizations such as GitLab, Alibaba, Xiaomi, Nintendo, and Adobe. Its progressive design allows organizations to integrate Vue into existing applications gradually or use it to build complete single-page applications from scratch.

Common Features of Component-Based Frameworks

Although React and Vue differ in syntax and ecosystem, they share several important features that support modern UI development.

The first shared feature is a component-based architecture. Both frameworks divide the interface into reusable components that encapsulate their own structure, behavior, and styling. This modular approach reduces duplicated code, simplifies maintenance, and allows developers to reuse interface elements across multiple pages (Freeman, Robson, Bates, & Sierra, 2022). A second shared feature is reactive state management. Both frameworks automatically update the user interface whenever application data changes. Instead of manually manipulating HTML elements, developers modify application state, and the framework efficiently refreshes only the necessary portions of the interface. This approach reduces programming errors and improves application performance. Another important feature is virtual DOM rendering. React introduced the virtual DOM concept, and Vue implements a similar optimization strategy. Rather than rebuilding the entire web page after every data change, both frameworks compare changes in memory before updating only the affected elements in the browser. This significantly improves responsiveness, especially for dashboards that frequently refresh business metrics.

Both frameworks also provide event handling and data binding. User interactions such as clicking buttons, selecting filters, or entering search criteria automatically trigger updates to application state. Vue offers two-way data binding through built-in directives, while React primarily uses one-way data flow with controlled components. Both approaches simplify user interaction while maintaining predictable application behavior. Finally, React and Vue both integrate easily with visualization libraries such as Chart.js, D3.js, and ApexCharts. This

capability is especially valuable for an analytics dashboard where business data must be displayed through interactive charts and graphs.

Comparison of Strengths and Limitations

React offers several advantages to this project. Its large ecosystem provides extensive libraries, tutorials, community support, and development tools. The framework is widely used in enterprise software development, making it easier to hire experienced developers and find solutions to technical problems. React also performs exceptionally well in large-scale applications that continue expanding over time (React, 2025). However, React also presents some challenges. Developers often need to choose separate libraries for routing, state management, form validation, and other common functionality. While this flexibility is valuable, it increases the number of architectural decisions required during development. For a newly formed team with limited time, configuring multiple libraries may slow initial development. Vue offers several strengths that closely align with the project requirements. Its syntax closely resembles traditional HTML, CSS, and JavaScript, making it easier for developers with web development experience to learn quickly. Vue includes many common features within its official ecosystem, reducing setup complexity and allowing teams to focus on application functionality rather than framework configuration (Vue.js, 2025). The primary limitation of Vue is its smaller ecosystem compared to React. Although Vue has grown considerably, React continues to dominate enterprise adoption, resulting in more third-party libraries, tutorials, and experienced developers. Organizations planning very large enterprise applications may benefit from React's broader community support.

Recommended Framework

For this scenario, Vue is the recommended framework. The primary reason is that the development team is small, newly formed, and working under a relatively short timeline. Vue's straightforward syntax, lower learning curve, and built-in ecosystem enable developers to become productive more quickly than React while still supporting modern component-based application design. Research indicates that Vue's progressive architecture allows organizations to develop interactive applications rapidly without sacrificing maintainability (Vue.js, 2025). Additionally, Freeman et al. (2022) emphasize that component-based design promotes modularity, improves code reuse, and simplifies long-term maintenance regardless of the chosen framework. These characteristics are particularly valuable for an analytics dashboard where reusable charts, filters, navigation, and reporting components can significantly reduce future development effort. Although React would be an excellent choice for very large enterprise applications with highly specialized development teams, Vue better satisfies this project's constraints by balancing performance, simplicity, maintainability, and development speed.

Conclusion

Selecting an appropriate front-end framework requires evaluating technical capabilities alongside project constraints. Both React and Vue support component-based development, reusable UI components, reactive state management, virtual DOM rendering, and integration with modern visualization libraries. React provides a larger ecosystem and greater enterprise adoption, while Vue offers a simpler learning experience and faster initial development. Considering the project's limited timeline, newly formed team, and need for maintainable code, Vue represents the most

practical solution. Its progressive architecture enables rapid development while providing the scalability needed as the analytics dashboard evolves.

References

Freeman, E., Robson, E., Bates, B., & Sierra, K. (2022). *Head First Design Patterns* (2nd ed.). O'Reilly Media.

React. (2025). *React documentation*. <https://react.dev>

Vue.js. (2025). *Vue.js documentation*. <https://vuejs.org>

Wieruch, R. (2022). *The Road to React* (2022 ed.). Self-published.